

Speculative Decoding

Deep Dive into SGLang — Chapter 17

Yin Yang

Foundation Books Project

Where this chapter sits

This is the chapter that makes **acceleration invisible when speculation is wrong**.

- Ordinary decoding pays one expensive target forward step per committed token.
- A cheap draft path proposes several future tokens; the target model verifies a block and often commits more than one token per step.
- The hard requirement is **exactness**: the visible token stream must match what target-only decoding would have produced.

Goal: by the end you can look at any proposed token and say whether it is draft evidence or target-verified request state.

Roadmap

The draft-and-verify problem

Tree verification and acceptance

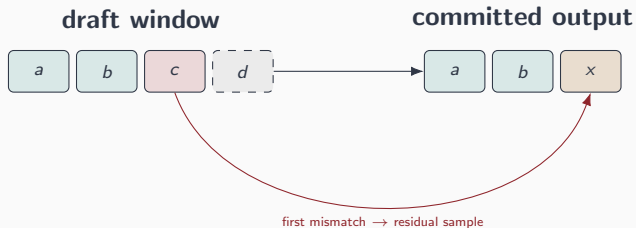
Correctness invariants

From concept to code

Measuring the trade

The draft-and-verify problem

One rejected token, never seen



Draft proposes $[a, b, c, d]$. The target accepts a, b , **rejects** c , samples replacement x . The request advances by $[a, b, x]$; c and d stay draft-owned.

Speculation is acceleration only while rejection remains invisible.

The accept/reject rule keeps the target law

At one unverified position, with target p_{tar} and draft p_{draft} , accept $\hat{y}$ with probability

$$\alpha(\hat{y}) = \min\left(1, \frac{p_{\text{tar}}(\hat{y})}{p_{\text{draft}}(\hat{y})}\right).$$

On rejection, resample from the positive residual

$$p_{\text{res}}(z) = \frac{(p_{\text{tar}}(z) - p_{\text{draft}}(z))_+}{\sum_v (p_{\text{tar}}(v) - p_{\text{draft}}(v))_+}.$$

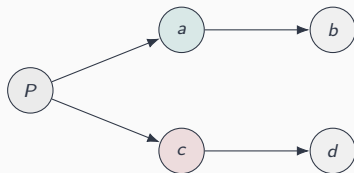
The two paths sum to $p_{\text{tar}}(z)$:

$$\underbrace{\min(p_{\text{draft}}, p_{\text{tar}})}_{\text{accept}} + \underbrace{(p_{\text{tar}} - p_{\text{draft}})_+}_{\text{reject}} = p_{\text{tar}}.$$

So the marginal committed token law equals target-only sampling.

Tree verification and acceptance

From a chain to a candidate tree



row *b* sees *P*, *a*
row *d* sees *P*, *c*
siblings hidden

A draft window is the one-branch case. A **candidate tree** spends one proposal budget on several continuations. *Tree attention* lets each candidate see only its committed prefix and its own ancestors.

The verifier commits a prefix of at most one root-to-leaf branch.

What a step commits

Draft window length k_{draft} , accepted draft length $a_r \in \{0, \dots, k_{\text{draft}}\}$:

- If $a_r < k_{\text{draft}}$: commit the accepted prefix *plus* a replacement, so $a_r + 1$ tokens.
- If $a_r = k_{\text{draft}}$: commit the whole window *plus* a target **bonus token**, up to $k_{\text{draft}} + 1$ tokens.

One target pass can advance the request by several committed tokens — that is the entire speedup mechanism.

Correctness invariants

The bookkeeping is where bugs live

Invariant (Target-verified prefix alignment)

*A speculative step is valid only when draft identity, target exactness, acceptance-prefix order, forward mode, cache materialization, lifetime, batch row, verifier policy, and repair state all describe the **same committed target prefix**.*

Most implementation bugs violate a bookkeeping invariant first: a stale position, a misaligned tree mask, or a padded speculative tensor not trimmed back to the real batch size.

From concept to code

From concept to code: the algorithm matrix

The scheduler's first question is *which proposer family*, and worker construction is mode-sensitive.

```
from python/sglang/srt/speculative/spec_info.py

class SpeculativeAlgorithm(Enum):
    DFLASH = auto(); EAGLE = auto(); EAGLE3 = auto()
    STANDALONE = auto(); NGRAM = auto(); NONE = auto()
    def is_eagle(self): # NOTE: EAGLE3 is a variant of EAGLE
        return self == ...EAGLE or self == ...EAGLE3
    def create_worker(self, server_args):
        if self.is_dflash() and enable_overlap:
            raise ValueError("DFLASH does not support overlap ...")
        elif self.is_eagle():
            ... EAGLEWorkerV2 if enable_overlap else EAGLEWorker
```

There is no EAGLE2 enum value: NEXTN is normalized to EAGLE before enum conversion.

Measuring the trade

Speedup must be weighed against wasted draft work

n-gram width	tok/s	Accept rate	Mean commit len	Rejected draft
2 draft tokens	3,623	44.2%	1.44	6,535
4 draft tokens	4,254	28.9%	1.87	19,300

Same Thor/Qwen workload. The four-token row improves throughput and committed length but lowers acceptance rate and triples rejected draft work.

A speculative speedup claim needs both an acceptance measurement and a materialization measurement.

What to carry forward

1. Speculative decoding is a **controlled bet**: draft proposes, target verifies, output commits only the target-accepted prefix.
2. The **accept/reject + residual** rule keeps the committed token law equal to the target law.
3. A draft window is the one-branch case of a **candidate tree** under tree attention.
4. Correctness is **bookkeeping**: identity, mode, cache, lifetime, and batch-row alignment all describe one committed prefix.
5. Every proposer family (EAGLE, DFlash, n-gram, standalone, MTP) is a **draft source** — not a different output law.

Next: Section 1 follows one rejected draft token from the draft window to the speculative commit boundary.